Respuesta de ejemplo · Actividad «Consultas SQL con un tutor de IA generativa»

Asignatura: Bases de Datos a Gran Escala (BDGE) · Máster en Big Data (UMU/USC) · curso 2026-27

Página de la actividad: dsevilla.github.io/bdge-ia-sql

> Documento de muestra. Recoge, ejercicio a ejercicio del itinerario, el enunciado tal y como lo plantea la página y la consulta final que cierra cada ejercicio (la solución de referencia del banco). Sirve de modelo de entrega y de guía de corrección.

1. Resumen: los diez ejercicios del itinerario

Nº	Ejercicio	En qué consiste	Estado
1	**Explora las tablas**	Muestra en orden alfabético las tablas y vistas de la base	cerrado
2	**Ordena preguntas por número de respuestas**	Obtén las diez preguntas con mayor AnswerCount	cerrado
3	**Resume las publicaciones por tipo**	Para cada PostTypeId calcula cuántas publicaciones hay y su puntuación media	cerrado
4	**Relaciona preguntas y autores**	Muestra diez preguntas con su puntuación y el DisplayName del autor	cerrado
5	**Encuentra preguntas sin respuesta**	Devuelve diez preguntas sin ninguna respuesta	cerrado
6	**Filtra un mes sin aplicar funciones a la columna**	Cuenta las preguntas creadas en marzo de 2021 y calcula su puntuación media redondeada a dos decimales	cerrado
7	**Resume las preguntas por mes con una CTE**	Usa una CTE para contar preguntas por mes y calcular su puntuación media	cerrado
8	**Encuentra las tres preguntas mejor puntuadas de cada año**	Usa ROW_NUMBER() con PARTITION BY año y filtra después las tres primeras de cada año	cerrado
9	**Construye y consulta un objeto JSON**	Construye un objeto JSON con id, título, puntuación y autor para cinco preguntas	cerrado
10	**Une cuatro tablas: pregunta, respuesta aceptada y los dos autores**	Para las preguntas cuya respuesta aceptada la escribió otro usuario, muestra el Id y el Title de la pregunta, el…	cerrado

2. Cuaderno de intentos

Cada ejercicio se cierra con un único intento correcto, así que no hizo falta preguntar a la IA; la última columna queda vacía. En una entrega real recogería el prompt enviado al modelo y su respuesta, junto con la comprobación de que la consulta resultante era correcta.

2.1 · Explora las tablas

Enunciado. Muestra en orden alfabético las tablas y vistas de la base. En DuckDB puedes consultar information_schema.tables; limita la salida al esquema main.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT table_type AS Tipo, table_name AS Nombre
FROM information_schema.tables
WHERE table_schema = 'main'
ORDER BY table_type, table_name;
```

2.2 · Ordena preguntas por número de respuestas

Enunciado. Obtén las diez preguntas con mayor AnswerCount. Muestra Id, Title y AnswerCount; desempata por Id ascendente.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT Id, Title, AnswerCount
FROM Posts
WHERE PostTypeId = 1
ORDER BY AnswerCount DESC, Id ASC
LIMIT 10;
```

2.3 · Resume las publicaciones por tipo

Enunciado. Para cada PostTypeId calcula cuántas publicaciones hay y su puntuación media. Conserva los grupos con al menos diez filas.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT PostTypeId,
       COUNT(*) AS NumeroPublicaciones,
       ROUND(AVG(Score), 2) AS PuntuacionMedia
FROM Posts
GROUP BY PostTypeId
HAVING COUNT(*) >= 10
ORDER BY PostTypeId;
```

2.4 · Relaciona preguntas y autores

Enunciado. Muestra diez preguntas con su puntuación y el DisplayName del autor. Conserva también las preguntas cuyo autor no tenga una fila referenciable en Users.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT p.Id, p.Title, p.Score, u.DisplayName
FROM Posts AS p
LEFT JOIN Users AS u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 1
ORDER BY p.Score DESC, p.Id ASC
LIMIT 10;
```

2.5 · Encuentra preguntas sin respuesta

Enunciado. Devuelve diez preguntas sin ninguna respuesta. Usa NOT EXISTS y ordena por fecha más reciente e Id para que el resultado sea determinista.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT q.Id, q.Title, q.CreationDate
FROM Posts AS q
WHERE q.PostTypeId = 1
  AND NOT EXISTS (
    SELECT 1
    FROM Posts AS a
    WHERE a.PostTypeId = 2
      AND a.ParentId = q.Id
  )
ORDER BY q.CreationDate DESC, q.Id DESC
LIMIT 10;
```

2.6 · Filtra un mes sin aplicar funciones a la columna

Enunciado. Cuenta las preguntas creadas en marzo de 2021 y calcula su puntuación media redondeada a dos decimales. Filtra con un rango semiabierto (CreationDate >= 1 de marzo y < 1 de abril) en lugar de aplicar month() o year() a la columna: así el filtro puede aprovechar un índice o las estadísticas del fichero.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT COUNT(*) AS Preguntas,
       ROUND(AVG(Score), 2) AS PuntuacionMedia
FROM Posts
WHERE PostTypeId = 1
  AND CreationDate >= TIMESTAMP '2021-03-01'
  AND CreationDate < TIMESTAMP '2021-04-01';
```

2.7 · Resume las preguntas por mes con una CTE

Enunciado. Usa una CTE para contar preguntas por mes y calcular su puntuación media. Muestra los 24 meses más recientes.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
WITH PreguntasPorMes AS (
  SELECT strftime(CAST(CreationDate AS TIMESTAMP), '%Y-%m') AS Mes,
         COUNT(*) AS NumeroPreguntas,
         ROUND(AVG(Score), 2) AS PuntuacionMedia
  FROM Posts
  WHERE PostTypeId = 1
    AND CreationDate IS NOT NULL
  GROUP BY strftime(CAST(CreationDate AS TIMESTAMP), '%Y-%m')
)
SELECT Mes, NumeroPreguntas, PuntuacionMedia
FROM PreguntasPorMes
ORDER BY Mes DESC
LIMIT 24;
```

2.8 · Encuentra las tres preguntas mejor puntuadas de cada año

Enunciado. Usa ROW_NUMBER() con PARTITION BY año y filtra después las tres primeras de cada año. Desempata por Id ascendente.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
WITH PreguntasPosicionadas AS (
  SELECT Id,
         Title,
         strftime(CAST(CreationDate AS TIMESTAMP), '%Y') AS Anio,
         Score,
         ROW_NUMBER() OVER (
           PARTITION BY strftime(CAST(CreationDate AS TIMESTAMP), '%Y')
           ORDER BY Score DESC, Id ASC
         ) AS Posicion
  FROM Posts
  WHERE PostTypeId = 1
    AND Title IS NOT NULL
    AND CreationDate IS NOT NULL
)
SELECT Anio, Posicion, Id, Title, Score
FROM PreguntasPosicionadas
WHERE Posicion <= 3
ORDER BY Anio DESC, Posicion ASC, Id ASC;
```

2.9 · Construye y consulta un objeto JSON

Enunciado. Construye un objeto JSON con id, título, puntuación y autor para cinco preguntas. Extrae la puntuación del documento con json_extract. Es una adaptación DuckDB del trabajo con JSON de la sesión.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT Id,
       json_object(
         'id', Id,
         'title', Title,
         'score', Score,
         'ownerUserId', OwnerUserId
       ) AS Documento,
       json_extract(
         json_object('score', Score),
         '$.score'
       ) AS PuntuacionExtraida
FROM Posts
WHERE PostTypeId = 1
ORDER BY Id ASC
LIMIT 5;
```

2.10 · Une cuatro tablas: pregunta, respuesta aceptada y los dos autores

Enunciado. Para las preguntas cuya respuesta aceptada la escribió otro usuario, muestra el Id y el Title de la pregunta, el DisplayName de su autor, el Id de la respuesta aceptada, el DisplayName de quien la escribió y la puntuación de la respuesta. Necesitas Posts dos veces (pregunta y respuesta) y Users dos veces (cada autor). Muestra las diez con la respuesta de mayor puntuación; desempata por Id de pregunta ascendente.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT q.Id AS PreguntaId,
       q.Title,
       uq.DisplayName AS AutorPregunta,
       a.Id AS RespuestaId,
       ua.DisplayName AS AutorRespuesta,
       a.Score AS PuntuacionRespuesta
FROM Posts AS q
JOIN Users AS uq ON uq.Id = q.OwnerUserId
JOIN Posts AS a ON a.Id = q.AcceptedAnswerId
JOIN Users AS ua ON ua.Id = a.OwnerUserId
WHERE q.PostTypeId = 1
  AND ua.Id <> uq.Id
ORDER BY a.Score DESC, q.Id ASC
LIMIT 10;
```

3. Cierre y reflexión

1. **Un error que la IA me ayudó a ver.** (Ejemplo) Confundía `WHERE` con `HAVING`: filtraba grupos antes de agregar y no entendía por qué el resultado no cuadraba.

2. **Una respuesta de la IA que descarté.** (Ejemplo) Me propuso `DATE_TRUNC` sobre una columna de texto; el comprobador seguía fallando y la documentación de DuckDB confirmó que había que convertir antes con `CAST`.

3. **Qué me llevo.** El comprobador no dice «está mal»: dice qué filas o columnas no cuadran, y eso es justo lo que hay que meter en el prompt.

4. Cómo se genera este PDF

```
make          # genera respuesta.pdf a partir de respuesta.md
make clean    # borra el PDF
make deps     # instala lo único necesario: reportlab
```

El conversor es `md2pdf.py` (solo requiere `reportlab`): entiende títulos, párrafos, listas, tablas, bloques de código y citas. No necesita LaTeX, pandoc ni conexión a red.